

Lab13. AWS Lambda – serverless service

Cloud Programming (W04IST–SI4527G)

Wojciech Thomas

Spring 2026

1 Learning objectives

1. Learn how to use serverless Lambda functions.

2 Exercise 1

In the first exercise you will create a Lambda function that will stop an EC2 instance. The function will be scheduled to be run every two minutes.

2.1 Create an EC2 instance

1. Create an EC2 instance. Verify that instance is running.

2.2 Create a Lambda function

1. Open list of AWS Services. From Compute choose Lambda.
2. Click “Create function” button.
3. On “Create function” select:
 1. Author from scratch.
 2. Function name: `myFun1`.
 3. Runtime: set current version of Python3.
 4. Expand “Change default execution role”.
 5. Select: “Use existing role”.
 6. From “Existing role” list select “LabRole”.
4. Click “Create function” button.

2.3 Configure trigger

1. Click “+ Add Trigger” button.
2. From drop down list select “EventBridge (CloudWatch Events)”.
3. Choose “Create a new rule” and set:
 - Rule name: `everyMinute`
 - Rule type: Schedule expression

- Schedule expression: `rate(2 minutes)`
4. Click “Add”

2.4 Define the function

1. Below the “Function overview” panel, select Code, and then open `lambda_function.py` in the left panel.
2. Delete the content of the “Code source” panel, and paste inside the following code:

```
import boto3
region = 'us-east-1'
instances = ['<instance-id>']
ec2 = boto3.client('ec2', region_name=region)

def lambda_handler(event, context):
    ec2.stop_instances(InstanceIds=instances)
    print('stopped your instances: ' + str(instances))
```

Warning: Do not remove apostrophes (') around region and instance id.

3. Make the following changes in the code:
 1. Open EC2 console, and find the ID of the running EC2 instance. Replace `<instance-id>` with the ID of your instance.
4. Click the “Deploy” button.
5. Switch from the “Code” tab to the “Monitor” tab.
6. In the box with**** time span (“1h 3h 12h ...”) click “Custom”, and select `15 min`. Observe the CloudWatch metrics.

2.5 Verification

1. Open EC2 console and verify the state of the EC2 instance. Refresh the view of list of instances.
2. If EC2 instance is stopped, start it, and observe its state for few minutes.

3 Exercise 2

In the second example you will create a Lambda function that process data passed with the event object. The result of the process will be returned as the result JSON file.

3.1 Processing events with Lambda functions

1. Open AWS Lambda panel and click “Create function” button.
2. Type the name `myFun2`.
3. Switch Runtime to current version of Python3.

4. Expand “Change default execution role”. Select “Use an existing role”. Select existing “LabRole”
5. Click “Create function” button.
6. Select the “Code” tab.
7. In the `lambda_function.py` change the function in the following way:

```
def lambda_handler(event, context):  
    print(event)  
    message = f"Hello {event.get('name', 'unknown')}!"  
    return {  
        'statusCode': 200,  
        'headers': {  
            'content-type': 'text/plain'  
        },  
        'body' : message  
    }
```

8. Click the “Deploy” button.

3.2 Send test events to the Lambda function

1. Select “Test” tab.
2. In “Test event action” select “Create new event”.
3. Set “Event name” to: `myEvent1`.
4. In the Event JSON add a key: `name` and set your name as a value.
5. Click the “Save” button above.
6. In “Event name” list select your event.
7. Click the “Test” button.
8. Expand “Executing function:succeeded” panel. Find the response received from the Lambda function.

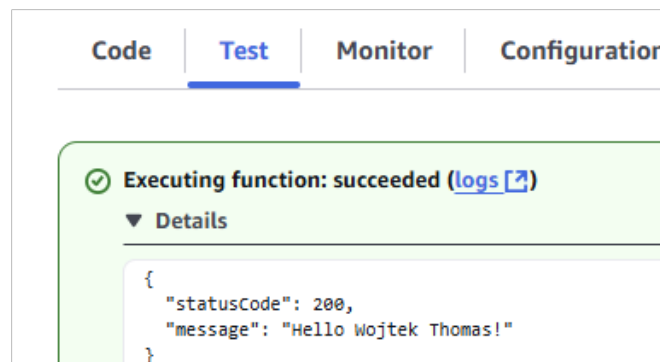


Figure 1: Lambda test result

9. Make a screenshot of the browser window, and paste it into assignment.

4 Exercise 3

In this exercise you will create a public API to access your Lambda function. This API will accept two types of requests: GET and PUT. In the first arguments will be passed in query string, in the latter, arguments will be passed in the body of the request as a JSON data.

4.1 Extracting arguments

Test module used in the previous step calls the Lambda function directly. If your function is called from Internet, the `event` dictionary had the different structure, including HTTP request parameters.

1. Change your function to extract body from the request, and convert it into JSON. Then read the parameter from the dictionary. Delete the line `message = f"Hello` and insert the following code:

```
body = event.get('body', '{}')
content = json.loads(body)
print(content)
message = f"Hello {content.get('name', 'unknown')}!"
```

2. Click “Deploy” button.
3. Switch to the “Test” tab. Create a new event. Name it `myEvent2`. In the Event JSON put the following code:

```
{
  "body": "{\"name\": \"<your-name>\"}"
}
```

Save the test.

4. Click the “Test” button.
5. Switch to “Configuration” tab, and select “Function URL”.
6. Click “Create function URL”. Select “Auth type” to NONE. Click “Save”.

You have just created a public endpoint to fire your Lambda function.

7. Start your Cloud9 environment. In the terminal type the following command:

```
curl -H 'content-type: application/json' \
  -d '{"name": "<your-name>"}' \
  <function-URL>
```

Replace `<your-name>` with your real name, and `<function-URL>` with URL created in the previous step.

You should see the greeting.

4.2 Create HTTP API trigger

Using function URL is quite limited. For enterprise applications, lambda functions are hidden behind API, that protect them and allow to create a facade for multiple lambda functions.

1. Click the “Add trigger” button.
2. Expand list “Select a source”. Select “API Gateway”. And configure:
 1. Intent: “Create a new API”,
 2. API type: “HTTP API”,
 3. Security: “Open”,
 4. Click “Add” button.
3. Select tab “Configuration”, and then “Triggers”.
4. Copy the API endpoint. It looks like:
`https://ww07u144576.execute-api.us-east-1.amazonaws.com/default/myFun2`
5. Get back to Cloud9. In the `curl` command, replace *Function URL* with *API endpoint*.

4.3 Create REST API trigger

HTTP API is very fast and very simple. However, if you need to make some transformation before the lambda function call, you can use more powerful REST API.

1. Click the “Add trigger” button.
2. Expand list “Select a source”. Select “API Gateway”. And configure:
 1. Intent: “Create a new API”,
 2. API type: “REST API”,
 3. Security: “Open”,
 4. Click “Add” button.
3. Select tab “Configuration”, and then “Triggers”. Expand Details sections, and find the REST API. Click the link `myFun2-API` next to **API Gateway**.
4. In “Resources” select method `ANY` under the `/myFun2` path. Click “Delete” button. Click again “Delete” to confirm remove.
5. Click “Create method” button, and configure:
 1. Method type: GET
 2. Integration type: Lambda function.
 3. Lambda function: click text area and select `myFun2` function.
 4. URL query string parameters: click “Add query string”, and set:
 1. Name: `name`
 2. Required: check the box.
 5. Click “Create method” button.
6. Click “Integration request” (between “Method request” and “Lambda integration”). Click “Edit” button.
 1. Select “Lambda function”
 2. In Lambda function select your `myFun2` function.

3. Expand “Mapping templates” and click “Add mapping template” button.
4. Set “Content type” to “application/json”
5. In the “Template body” insert:

```
{  
  "name": "$input.params('name')"  
}
```

Click “Save” button.

7. Click “Deploy API” button. Select default Stage and click “Deploy” again.
8. In Stages expand `default`, `/`, `myFun2`, `GET`, and copy Invoke URL.
9. Paste the Invoke URL into browser address bar, and append string: `?name=<your-name>`. Press Enter and watch what is displayed in the browser.
10. In Cloud9 terminal run command `curl`:
`curl https://e4d...aws.com/default/myFun2?name=<your-name>`

Can you see the greeting?